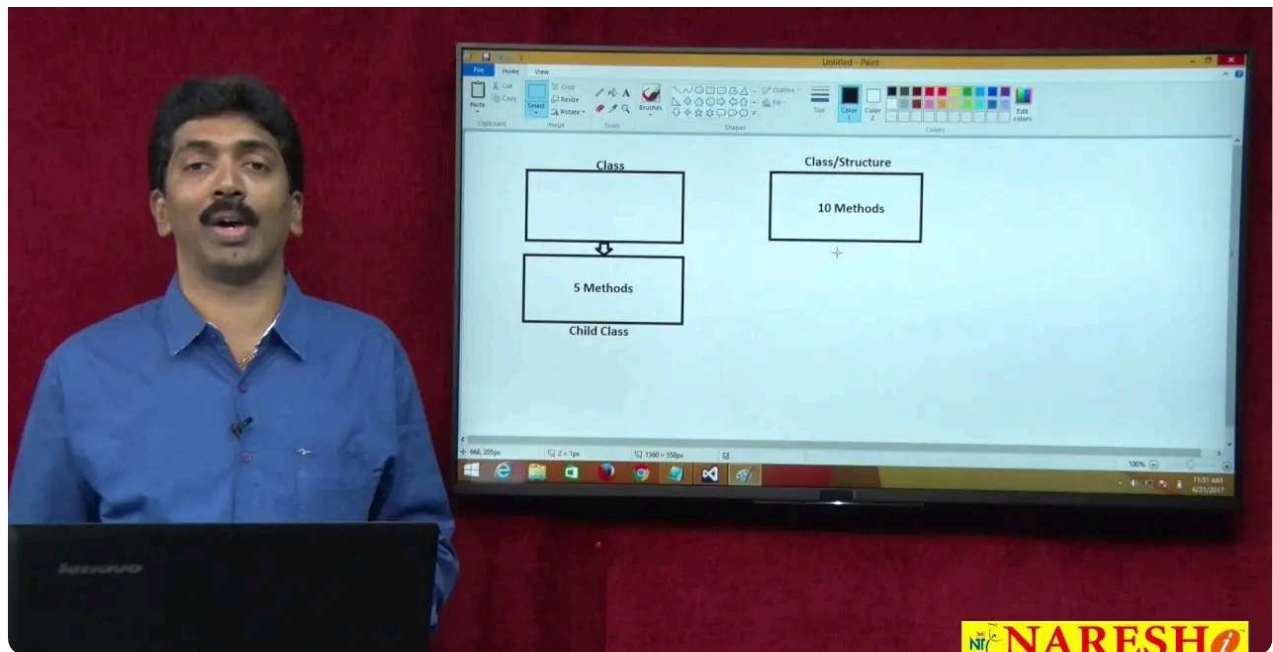




Understanding Extension Methods in C#.NET

By Albert Harmon Published September 30, 2017 ⌚ 4 min read [Technology](#)



TLDR Extension methods in C#.NET allow developers to add new methods to existing classes or structures without modifying their source code. Introduced in C# 3.0, these methods provide a way to enhance functionality while avoiding common issues

associated with inheritance, such as sealed classes and lack of permissions.

In this blog post, we will explore extension methods in C#.NET, a powerful feature introduced in C# 3.0. Extension methods allow developers to add new methods to existing classes or structures without modifying their source code. This capability is particularly useful when the source code is unavailable or when modifications would require extensive retesting.

What Are Extension Methods?

Extension methods are a mechanism that enables developers to add new methods to existing types, such as classes or structures, without needing access to their source code. This feature is beneficial in various scenarios, including:

- When the source code of a class is not available.
- When the class is sealed, preventing inheritance.
- When modifications to the class would require retesting, which can be time-consuming.

The Need for Extension Methods

Consider a scenario where you have a class with ten methods, and you want to add five more. If you do not have the source code or

permission to modify the class, you face several challenges:

1. **Missing Source Code:** You cannot add methods to a class if you do not have its source code. For example, you cannot modify the built-in `string` class in .NET.
2. **Permissions:** If the class has already been tested and deployed, any changes would require retesting, which is often impractical.

Inheritance as a Solution

One common approach to extend functionality is through inheritance. By creating a child class, you can inherit existing methods and add new ones. However, this approach has limitations:

- **Sealed Classes:** If the class is sealed, inheritance is not possible.
- **Structures:** You cannot inherit from structures, as they do not support inheritance.

How Extension Methods Work

To overcome the limitations of inheritance, C# introduced extension methods. Here's how they work:

1. **Static Class:** Define a static class to contain the extension methods.
2. **Static Methods:** All methods within this static class must also be static.

3. **Binding:** The first parameter of each method specifies the type to which the method is bound. This parameter is used for binding and is not passed when calling the method.

Example of Creating an Extension Method

Let's say we have a class named ``Program`` with two methods. We want to add a third method without modifying the original class:

```
public class Program
{
    public void TestOne() { Console.WriteLine("Method One"); }
    public void TestTwo() { Console.WriteLine("Method Two"); }
}
```



To add a new method, we create a static class:

```
public static class ExtensionMethods
{
    public static void TestThree(this Program program)
    {
        Console.WriteLine("Method Three");
    }
}
```

Now, we can call `TestThree` on an instance of `Program` :

```
Program p = new Program();  
p.TestOne();  
p.TestTwo();  
p.TestThree(); // Calls the extension method
```

Important Points to Remember

1. **Static Methods:** Extension methods are defined as static but can be called as if they were instance methods of the class they extend.
2. **Name Conflicts:** If an extension method has the same name and signature as an existing method, the original method will be called instead of the extension method.
3. **Single Binding Parameter:** An extension method can only have one binding parameter, which must be the first parameter in the method signature.
4. **No Source Code Modification:** Extension methods do not require access to the original class's source code or permissions to modify it.

Extending Structures and Sealed Classes

Extension methods can also be applied to structures and sealed classes. For example, you can add a method to the `int` structure:

```
public static class IntExtensions
{
    public static long Factorial(this int number)
    {
        if (number <= 1) return 1;
        return number * (number - 1).Factorial();
    }
}
```

You can then call this method on an integer:

```
int num = 5;
long result = num.Factorial(); // Calls the extension method
```



Conclusion

Extension methods in C#.NET provide a flexible way to enhance existing classes and structures without modifying their source code. This feature is particularly useful in scenarios where source code is unavailable or when modifications would require extensive retesting. By understanding how to implement and use extension methods,

developers can significantly improve their productivity and code maintainability.